GNN Drug-Drug Interaction

Francesco Virgulti

August 2025

1 Introduction

Graph Neural Networks (GNNs) are a class of neural networks specifically designed for tasks where the input data is structured as a graph. Their development was inspired by Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs). While CNNs and RNNs typically operate on grid-like data such as images or sequences—where each element is connected to at most eight neighbors—in graphs, a single node can potentially connect to any number of other nodes, with various types of edges defining those relationships.

The objective of this work is to predict potential interactions between pairs of drugs, each of which can be naturally represented in the form of a graph.

2 Graphs

A graph is a mathematical structure used to represent a set of objects and the relationships between them. The objects are called nodes (or vertices), and the connections between them are called edges. Formally, a graph G is defined as a tuple:

$$G = (V, E)$$

where V is the set of nodes and E is the set of edges. The number of nodes in a graph is denoted as $N = |V|$. Graphs can be directed or undirected, depending on whether the edges have a direction. In the case of molecular graphs, edges correspond to chemical bonds, which are typically undirected, as the bond is mutual between two atoms.

A common way to represent a graph is through its adjacency matrix A, an $N \times N$ matrix, where each entry indicates whether an edge exists between a pair of nodes.

3 From Drug to Graph

Biologically, drugs are chemical compounds composed of atoms connected by chemical bonds. Each drug can be represented by its molecular structure, typi-

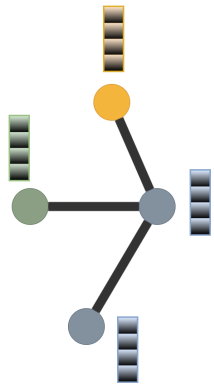


Figure 1: Drug Schema

cally encoded as a SMILES string. Using cheminformatics tools such as RDKit, this molecular structure can be translated into a graph, where:

- Each **node** corresponds to an atom in the drug molecule.
- Each **edge** corresponds to a chemical bond between atoms.

Mathematically, the molecular graph is defined by its adjacency matrix A and its feature matrix X. The adjacency matrix encodes the connectivity of atoms, while the feature matrix characterizes the properties of each atom. In our model, the feature matrix is $X \in \mathbb{R}^{N \times F}$, where $F = 17$ features are considered for each atom:

1. **Atomic number**: the number of protons in the nucleus of the atom. (`a.GetAtomicNum()`)
2. **Degree**: the number of covalent bonds connected to the atom. (`a.GetDegree()`)
3. **Total number of hydrogens**: both implicit and explicit hydrogens attached to the atom. (`a.GetTotalNumHs()`)
4. **Aromaticity**: whether the atom belongs to an aromatic system (binary). (`int(a.GetIsAromatic())`)
5. **Formal charge**: the formal charge of the atom. (`a.GetFormalCharge()`)
6. **Total valence**: the total valence of the atom. (`a.GetTotalValence()`)
7. **Explicit valence**: the valence contributed by explicitly drawn bonds. (`a.GetExplicitValence()`)
8. **Implicit valence**: valence contributed by implicit hydrogens or electrons. (`a.GetImplicitValence()`)
9. **Radical electrons**: the number of unpaired electrons. (`a.GetNumRadicalElectrons()`)

10. **Ring membership**: whether the atom is part of any ring (binary). (`int(a.IsInRing())`)
11. **Ring size 3**: whether the atom is in a 3-membered ring. (`int(a.IsInRingSize(3))`)
12. **Ring size 4**: whether the atom is in a 4-membered ring. (`int(a.IsInRingSize(4))`)
13. **Ring size 5**: whether the atom is in a 5-membered ring. (`int(a.IsInRingSize(5))`)
14. **Ring size 6**: whether the atom is in a 6-membered ring. (`int(a.IsInRingSize(6))`)
15. **Hybridization**: the hybridization state of the atom (encoded as integer). (`int(a.GetHybridization())`)
16. **Chirality tag**: the stereochemical configuration of the atom (enum encoded as integer). (`a.GetChiralTag()`)
17. **Scaled atomic mass**: atomic mass multiplied by 0.01 (for numerical stability). (`a.GetMass() * 0.01`)

4 Graph Convolutional Layer

In a Graph Convolutional Network (GCN) layer, the input feature matrix X is multiplied by the normalized adjacency matrix $\hat{A}$, yielding:

$$X^{(k+1)} = \hat{A}\, X^{(k)} W^{(k)},$$

where $W^{(k)} \in \mathbb{R}^{F_{\text{in}} \times F_{\text{out}}}$ is a learnable weight matrix at layer k, and $\hat{A}$ is the symmetrically normalized adjacency matrix:

$$\hat{A} = D^{-\frac{1}{2}}(A + I)D^{-\frac{1}{2}},$$

with D the diagonal degree matrix and I the identity matrix ensuring self-loops. This operation aggregates information from neighboring atoms to update node representations.

5 Model

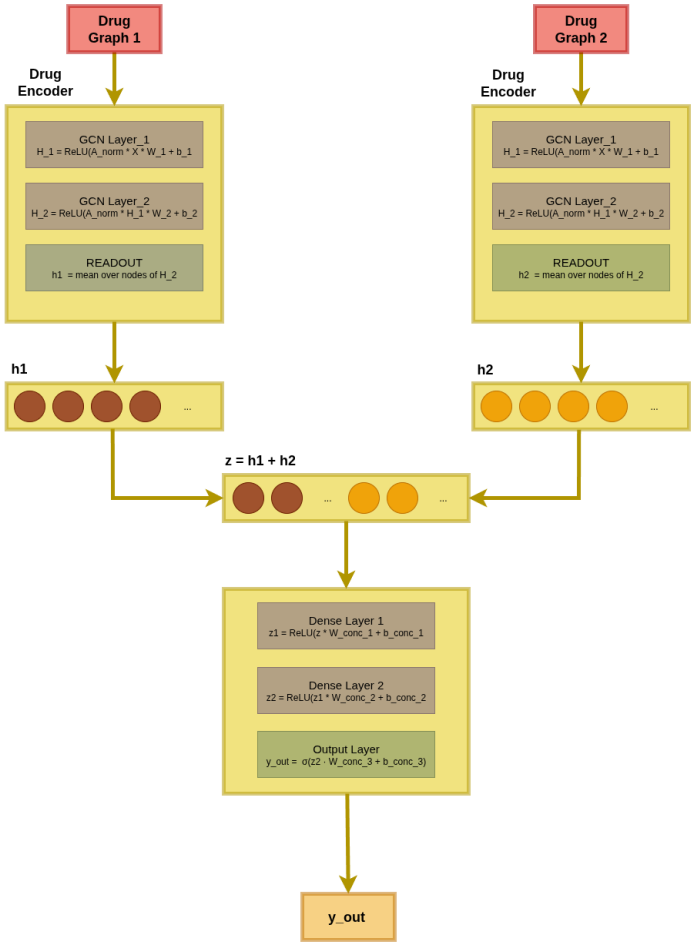


Figure 2: Model Description

Our model takes two drug graphs as input, each represented by its adjacency matrix and node features. Both drugs are processed independently through a GCN-based encoder, consisting of two graph convolution layers with ReLU activation, followed by a mean pooling operation to obtain a fixed-size embedding (h_1 and h_2). These embeddings are then concatenated to form a joint representation z, which is passed through a stack of dense layers (with ReLU activations) to progressively refine the features. Finally, a sigmoid output layer produces a scalar value y_{out}, representing the predicted probability of interaction between the two drugs.

5.1 Iterative Solver

The optimization of the loss function is carried out using *Stochastic Gradient Descent* with mini-batching. To regulate the *learning rate* and promote stable convergence, a *cosine decay schedule* is employed, progressively reducing the rate from 10^{-1} to 10^{-4} over the course of training.

5.2 Layer Size

The dimensionality of each layer is determined based on the size of the training dataset. Specifically, when the number of training samples is less than 1000, the drug encoder parameter dimension is set to 128; for larger datasets, this dimension is increased to 512.

The dimensionality of the *merge layer* is fixed : both *dense layers* are configured with a size of 512.

6 Dataset

The original dataset associated with this project was unavailable, as it relied on an outdated version of **PyTorch** that could not be installed on the computational device. As a replacement, the **TDC DrugBank** dataset was employed, which provides only **positive** drug–drug interactions (i.e., pairs with confirmed true labels).

To construct **negative** interactions, random drug pairs absent from the dataset were generated. This strategy is widely adopted in the literature; however, it introduces potential limitations, as such pairs may in fact correspond to interactions that exist but have not yet been experimentally validated.

This restricts the accuracy of the neural network as it is trained to classify certain pairs as negative despite their feature representations being potentially indistinguishable from those of true interactions.

6.1 GPU Improvement

Each drug in the dataset is represented as a graph, encoded by an adjacency matrix of size *Number_Atoms* $\times$ *Number_Atoms*. Since drugs contain a variable number of atoms, the resulting matrix dimensions are not uniform. This variability poses a challenge when transferring computation from the CPU to the GPU, as inconsistent memory allocation leads to reduced computational efficiency.

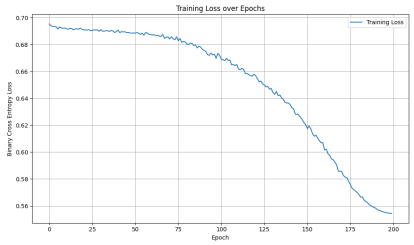
A straightforward solution is to standardize the matrix dimensions by fixing them to the size of the largest molecule in the dataset. To achieve this, all drugs are preprocessed, and the maximum number of atoms observed is recorded prior to converting the graphs into matrix form.

7 Results

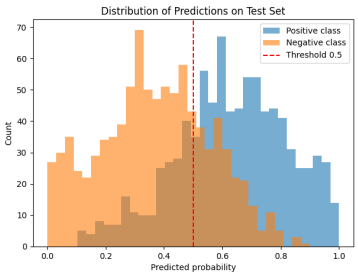
In this section, we evaluate the performance of the model for different maximum numbers of nodes (N), corresponding to the largest number of atoms per drug graph considered. Table 1 reports the accuracy, dataset size, and training time for $N = 50, 40$, and 30. Figures 3–5 show the corresponding training loss curves and prediction distributions on the test sets.

Table 1: Summary of results for different maximum numbers of nodes (N).

N	Accuracy (%)	Training Samples	Test Samples	Training Time
50	72	6235	1784	27 min
40	70	5812	1627	24 min
30	60	3981	1125	15 min

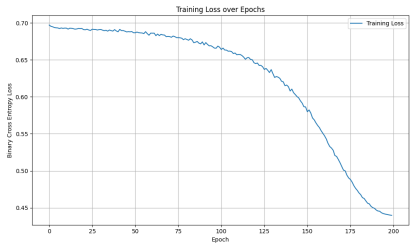


(a) Training loss ($N = 50$).

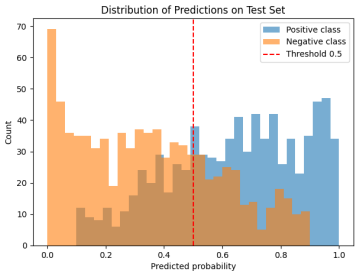


(b) Prediction distribution on test set ($N = 50$).

Figure 3: Results for $N = 50$.

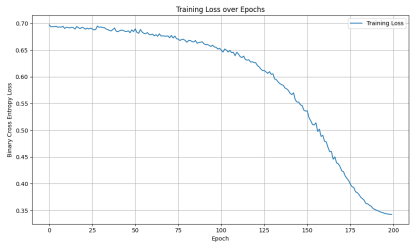


(a) Training loss ($N = 40$).

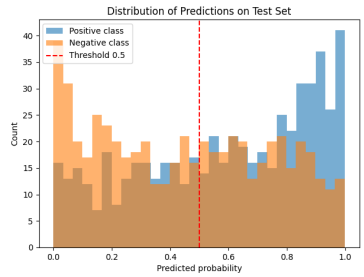


(b) Prediction distribution on test set ($N = 40$).

Figure 4: Results for $N = 40$.



(a) Training loss ($N = 30$).



(b) Prediction distribution on test set ($N = 30$).

Figure 5: Results for $N = 30$.

8 Conclusion

Overall, the neural network was able to identify the principal features underlying drug–drug interactions, despite the limitations in the construction of the dataset. Moreover, this work could be further extended by incorporating analyses that go beyond the purely mathematical representation of drugs. In particular, integrating biological information, such as the Minimum Common Structure between two compounds or their complete three-dimensional conformations, may provide a more comprehensive understanding of drug–drug interactions.